Instruction formats

32-bit fixed width. Opcode is always bits [31:26]. Numbers mark field boundaries (MSB left).

R register ops

I two regs + immediate

U one reg + large immediate

C control-register access

N no operands

R: rd←rs1 op rs2; MOV/NOT ignore rs2; BALR uses rd=link, rs1=target. **I**: load/store r1=reg, r2=base, imm=offset; branch r1,r2=operands, imm=signed PC-rel (±512 KB). **U**: BAL imm=signed PC-rel (±8 MB). **C**: cr=index 0–3.

Opcodes

data & arithmetic			*control & privilege*		
0x02	MOV	R	0x00	HLT	N
0x03	LDI16	U	0x01	NOP	N
0x04	LDI32L	U	0x16	BEQ	I
0x05	LDI32H	U	0x17	BNE	I
0x06	LDM8	I	0x18	BLT	I
0x07	LDM16	I	0x19	BGE	I
0x08	LDM32	I	0x1A	BLTU	I
0x09	STR8	I	0x1B	BGEU	I
0x0A	STR16	I	0x1C	BAL	U
0x0B	STR32	I	0x21	BALR	R
0x0C	ADD	R	0x1D	ECALL	N
0x0D	SUB	R	0x1E	ERET	N
0x0E	MUL	R	0x1F	MFCR	C
0x0F	AND	R	0x20	MTCR	C
0x10	OR	R			
0x11	XOR	R			
0x12	NOT	R			
0x13	SLL	R			
0x14	SLR	R			
0x15	SAR	R			

Opcodes 0x22–0x3F (34–63) reserved.

Registers

r0	scratch / syscall # / return value (volatile)
r1-r3	arguments (caller-saved)
r4-r7	temporaries (caller-saved)
r8-r12	callee-saved (push on entry, pop before RET)
r13 SP	stack pointer — full descending, 4-byte aligned
r14 LR	link register — return address (caller-saved)
r15 PC	program counter (not writable)

Control registers — MFCR/MTCR, index 0–3: tvec 0, epc 1, cause 2, mode 3. CPU boots in supervisor mode; SP=0x1FFC at boot.

Memory map

0x0000-0x0FFF	kernel code + data
0x1000-0x11FF	kernel disk scratch
0x1200-0x1FFB	kernel stack
0x2000-	user program
0xFFFF0000	serial TX — write byte
0xFFFF0004	serial RX — read byte (blocks)
0xFFFF0010	disk sector register
0xFFFF0014	disk buffer register
0xFFFF0018	disk command (0 = read sector)

No memory protection — user code may touch any address.

Traps

ECALL: epc←PC, cause←0, mode←sup, PC←tvec.
ERET: mode←user, PC←epc.
No registers saved automatically — the handler does it.

Syscalls

Number in r0, args in r1-r3, return in r0.

1	write	r1=buf ptr, r2=byte count
2	read	returns byte in r0 (blocks)

Pseudo-instructions

LI rd,imm	LDI32L + LDI32H
PUSH rs	SP -= 4; STR32 rs,[SP]
POP rd	LDM32 rd,[SP]; SP += 4
CALL lbl	BAL r14,lbl (clobbers r14)
RET	BALR r0,r14
JMP lbl/rs	BAL r0,lbl / BALR r0,rs
ADDI rd,imm	LDI16 r0,imm; ADD rd,rd,r0

LI clobbers nothing, CALL clobbers r14, the rest clobber r0. CALL/RET do no stack traffic: a non-leaf function does push lr first and pop lr before RET (same for r8-r12 if used); leaf functions need neither.

BlissFS

Block = 512 B (one sector). Magic 0xB2155F2D. 32 inodes, filename ≤24 B, file ≤4096 B (8 × 512).

Disk layout: blk 0 superblock · blk 1–4 inode table (32 × 64 B) · blk 5 free-block bitmap · blk 6+ data.

Superblock (20 B): magic · block size · inode count · data-start block (6) · free-block count — 4 B each.

Inode (64 B): size(4) · type(1: 0 unused / 1 file / 2 dir) · resv(3) · 8 direct block pointers (32) · resv(24). Inode 0 null, inode 1 = root dir.

Dir entry (28 B): filename(24, null-padded) · inode number(4, 0 = empty). 18 per block.